

Product Requirements Document — Intenti condivisi

Executive summary

Intenti condivisi è il livello collaborativo della fase di pianificazione: mentre ogni giocatore prepara movimento, abilità e interazioni, i compagni vedono in tempo reale una rappresentazione privata del piano; gli avversari non ricevono né visualizzano questi dati. La feature comprende traiettorie, aree d'effetto, bersagli, ghost preview, etichette tattiche, stato di preparazione, notifiche, disegno temporaneo, cronologia, rollback e avvisi di conflitto.

La feature ha un valore centrale per il gioco, non puramente cosmetico. La ricerca sui workspace collaborativi mostra che informazioni visive condivise aiutano a mantenere consapevolezza dell'attività altrui, stabilire un contesto comune e coordinare le azioni; ritardare gli aggiornamenti visivi riduce parte di questi benefici. ¹ Il pattern è inoltre coerente con giochi tattici basati sulla pianificazione: *Frozen Synapse* permette di costruire e provare ordini prima dell'esecuzione simultanea, mentre *Phantom Brigade* usa una timeline predittiva per orchestrare azioni coordinate. *Atlas Reactor* basava il proprio ritmo su turni simultanei nei quali i giocatori dovevano coordinarsi e bloccare rapidamente le mosse.

²

La soluzione raccomandata è:

1. **Preview locale immediata**, senza attendere il server.
2. **Server autoritativo**, che valida ogni intenzione e ne assegna la revisione canonica.
3. **Distribuzione esclusivamente agli alleati**, tramite Client RPC sui rispettivi `PlayerController` o tramite componenti replicati owner-only.
4. **RPC unreliable per gli aggiornamenti frequenti e reliable per conferma, rollback e cambio di stato**.
5. **Snapshot periodici o Fast Array owner-only** per recuperare aggiornamenti persi.
6. **Nessun `NetMulticast` contenente intenzioni private**, perché un multicast viene eseguito sulle macchine per le quali l'attore è rilevante e non rappresenta, da solo, una separazione sicura fra squadre. ³
7. **Modding data-driven come ultima feature funzionale**, dopo che protocollo, schema dati e regole di sicurezza sono stabilizzati.

Per un team da quattro contro quattro, la feature non richiede netcode deterministico completo o rollback della simulazione. Il "rollback" previsto dal PRD riguarda le revisioni del piano durante la fase decisionale. Sistemi come GGPO o il Network Prediction Plugin salvano, ripristinano e risimulano lo stato del gioco e sarebbero eccessivi per il solo scambio degli intenti; potranno essere valutati separatamente per la risoluzione del turno. ⁴

Assunzioni di progetto

Voce	Assunzione
Modalità principale	PvP quattro contro quattro
Modello di rete	Dedicated server autoritativo
Durata pianificazione	Circa 20–30 secondi, configurabile
Engine	Unreal Engine 5.7 o 5.8, versione bloccata per ogni release
Piattaforma iniziale	PC
Sviluppo	Un programmatore principale, forte in C# e in apprendimento su C++
Persistenza	Nessun database nel percorso critico del match
Modding	Data-only inizialmente; codice nativo solo per mod curate
Stime	Giorni-persona netti, esclusi asset grafici definitivi e localizzazione completa

Obiettivi, benefici e requisiti

Gli obiettivi primari sono rendere leggibile la strategia degli alleati, ridurre la dipendenza da chat vocale, prevenire errori evitabili e conservare la tensione derivante dall'incertezza sulle mosse nemiche. Il sistema non deve trasformarsi in un simulatore perfetto del futuro: mostra ciò che gli alleati **intendono** fare, non garantisce ciò che accadrà dopo la risoluzione simultanea.

La distinzione è importante. Una traiettoria alleata deve essere presentata come piano provvisorio; l'interfaccia non deve suggerire che collisioni, spostamenti forzati, distruzione della mappa o azioni nemiche siano già risolti. La preview di *Frozen Synapse*, per esempio, permette di testare piani ipotetici senza conoscere il piano reale dell'avversario; *Phantom Brigade* usa invece la previsione come meccanica esplicita del gioco. Il nostro sistema segue il primo approccio: **condivisione dell'intento alleato, non previsione certa del turno.** ⁵

Obiettivi misurabili

- Un alleato deve capire in meno di due secondi dove un compagno vuole muoversi, quale area vuole colpire e se ha confermato.
- I giocatori devono poter correggere un piano senza interrompere il flusso degli altri.
- Gli errori più comuni — stessa cella finale, fuoco amico previsto, risorsa condivisa duplicata — devono essere segnalati prima del lock.
- Nessun dato degli intenti avversari deve raggiungere il client nemico.
- Il sistema deve restare utile anche senza chat vocale.
- La struttura dati deve supportare nuovi tipi di intento aggiunti tramite mod.

Fuori ambito iniziale

Non rientrano nel primo rilascio la previsione delle mosse nemiche, un replay deterministico completo del combattimento, testo libero nelle etichette, scripting arbitrario scaricato da server non affidabili e risoluzione automatica dei conflitti tattici.

Requisiti funzionali

ID	Requisito	Comportamento richiesto	Priorità	Stima
FR-TRAJ	Traiettorie	Mostrare spline o sequenza di celle, cambi di livello, punti di transizione e destinazione finale	P0	3-5 giorni
FR-AOE	Aree d'effetto	Visualizzare cerchio, cono, linea, anello o forma custom dell'abilità dalla posizione prevista	P0	3-5 giorni
FR-TARGET	Bersagli	Evidenziare personaggio, cella, oggetto o elemento della mappa scelto	P0	2-3 giorni
FR-LABEL	Etichette di intento	Associare tag predefiniti come Focus, Protezione, Scout, Escape, Trappola	P0	1-2 giorni
FR-STATE	Stato giocatore	Mostrare Editing, Ready, Locked, Disconnected e AFK/Timeout	P0	2-3 giorni
FR-CONFIRM	Conferma e lock	Permettere conferma, annullamento prima del lock e blocco immutabile allo scadere	P0	3-5 giorni
FR-NOTIFY	Notifiche	Segnalare cambio bersaglio, annullamento ultimate, conferma e rollback senza spam	P1	2-4 giorni
FR-GHOST	Simulazione ghost	Mostrare posizione prevista, orientamento, stance e volume dell'azione senza muovere l'attore reale	P0	5-8 giorni
FR-CONFLICT	Avvisi conflitto	Rilevare collisioni, fuoco amico, celle contese, risorse duplicate e azioni incompatibili	P0	8-12 giorni
FR-DRAW	Disegno sulla mappa	Tratti e frecce temporanei team-only con limiti, timeout e mute	P1	4-6 giorni
FR-HISTORY	Cronologia modifiche	Conservare una breve sequenza di revisioni e una timeline sintetica dei cambiamenti importanti	P1	3-5 giorni
FR-ROLLBACK	Rollback del piano	Ripristinare una revisione precedente generando una nuova revisione canonica	P1	4-6 giorni
FR-RECONNECT	Riconnessione	Inviare snapshot completo degli intenti visibili e stato del turno dopo reconnect	P1	3-5 giorni

ID	Requisito	Comportamento richiesto	Priorità	Stima
FR-FILTER	Filtri e accessibilità	Nascondere singoli overlay, cambiare intensità, usare pattern oltre al colore e supportare color blindness	P1	3-5 giorni
FR-MOD	Intenti moddabili	Registrazione etichette, renderer e tipi di intento data-driven, con schema versionato	P2, ultima feature	10-15 giorni

La stima netta dei requisiti è circa **58-89 giorni-persona**, prima del buffer di integrazione. Alcune attività procedono in parallelo: per esempio traiettorie, AOE e ghost condividono gran parte del renderer.

Requisiti non funzionali

Area	Requisito
Latenza	Risposta locale entro un frame; aggiornamento alleato mediano sotto 100 ms e p95 sotto 200 ms quando l'RTT è inferiore o uguale a 120 ms
Sicurezza	Tutti i payload client devono essere validati dal server; target, turno, ownership, dimensione, frequenza e coordinate devono essere controllati
Privacy tattica	Il server non deve inviare intenzioni, revisioni, target o identificatori nascosti ai client nemici
Affidabilità	Conferma, lock, rollback e snapshot devono convergere anche con perdita di pacchetti
Scalabilità	Target iniziale di otto giocatori; design compatibile con sedici senza cambiare il protocollo
Banda	Massimo tipico 5 KB/s per client per la feature; picco consigliato inferiore a 10 KB/s
Frequenza	Preview limitata a 10-12 aggiornamenti al secondo per giocatore; rendering locale a frame rate pieno
Osservabilità	Telemetria di latenza, dimensione payload, revisioni scartate, conflitti e fallimenti di validazione
Modding	Schema versionato, whitelist server, hash dei pacchetti e fallback per tipi sconosciuti
Privacy utente	Disegni e cronologia non persistenti oltre il match salvo replay o telemetria esplicitamente configurata
Accessibilità	Informazioni non codificate esclusivamente tramite colore; supporto a opacità, spessore, icone e pattern
Compatibilità	Client e server devono negoziare versione del protocollo e manifest delle mod

Unreal utilizza un modello client-server nel quale il server conserva lo stato autoritativo; le proprietà replicate vengono inviate dal server ai client e le Server RPC possono essere validate secondo una politica "trust and verify". Questi meccanismi sono appropriati per impedire che il client dichiari autonomamente una mossa valida o decida a chi distribuirla. ⁶

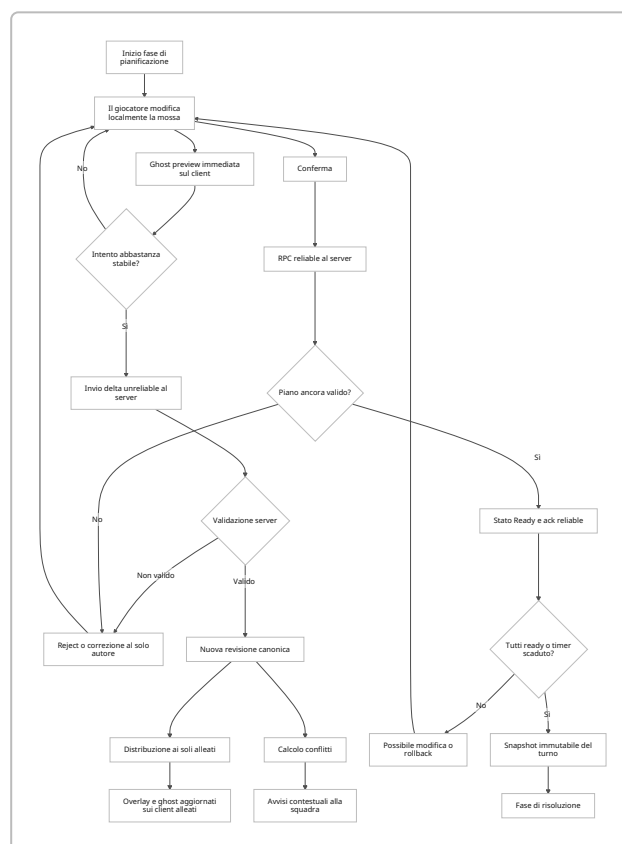
Esperienza utente e flussi

Durante la pianificazione, ogni giocatore vede quattro livelli distinti:

Livello	Esempi	Visibilità
Piano locale non inviato	Hover, mirino in movimento, path incompleto	Solo autore
Intento condiviso provvisorio	Percorso, AOE, bersaglio, etichetta	Autore e alleati
Intento confermato	Stesso overlay con stato Ready	Autore e alleati
Piano bloccato	Snapshot immutabile usato dal resolver	Server e squadra; nessuna modifica

Questa separazione evita di trasmettere ogni movimento del mouse. L'utente può esplorare localmente; l'intento viene condiviso quando esiste una forma valida o dopo un breve debounce, per esempio 80-100 ms.

L'overlay di ogni compagno deve avere un'identità visiva stabile: icona del personaggio, nome abbreviato, pattern, spessore della linea e colore. Il colore da solo non è sufficiente. Quando più traiettorie si sovrappongono, la UI deve applicare offset laterali, priorità all'intento selezionato e dissolvenza degli intenti non rilevanti.



Flusso di pianificazione di squadra. Il giocatore seleziona una destinazione. Il client calcola il path, disegna immediatamente la spline e invia una versione compressa al server. Gli alleati vedono il percorso e la cella finale; se viene scelta un'abilità, appaiono bersaglio e AOE calcolati dalla posizione prevista.

Modifica in tempo reale. Ogni cambiamento incrementa `Revision`. Gli update possono arrivare fuori ordine perché sono unreliable: il ricevente applica solo revisioni maggiori di quella già visualizzata. Un update perso non richiede ritrasmissione immediata, perché il successivo contiene lo stato aggiornato o un delta riferito a una base esplicita.

Conferma. Premendo "Pronto", il client invia una richiesta reliable. Il server ricostruisce o ricontrolla il percorso, valida abilità e bersaglio, assegna lo stato `Ready` e invia un acknowledgement. L'icona passa da "editing" a "ready"; il piano può restare modificabile fino al lock, ma una successiva modifica riporta automaticamente il giocatore in `Editing`.

Rollback. La UI presenta "Annulla modifica" e una breve cronologia. Il rollback non riutilizza il vecchio numero: crea una nuova revisione che copia il contenuto della revisione selezionata. In questo modo la sequenza resta monotona e i pacchetti ritardati non possono sovrascrivere il piano ripristinato.

Timeout. Alla scadenza, il server blocca l'ultima revisione confermata. Se non esiste, applica una regola esplicita: mantenere posizione, usare una mossa salvata localmente e già accettata dal server, oppure selezionare `Hold`. Il client non può inviare modifiche dopo il lock.

Conflitti. Gli avvisi sono consultivi, salvo invalidità formali. Due personaggi che raggiungono la stessa cella possono essere un errore oppure una strategia intenzionale: la UI segnala il problema, ma la risoluzione segue le regole del gioco. Un'abilità senza linea di tiro o un percorso fuori mappa è invece un errore bloccante.

Gli avvisi devono essere aggregati:

▲ Collisione prevista
Echo e Nova terminano nella stessa cella, livello 2.

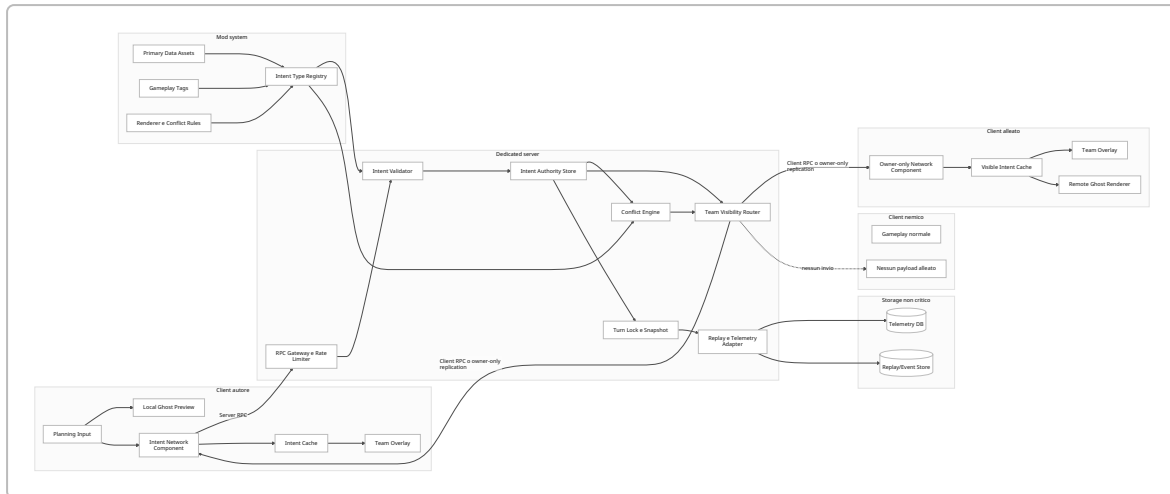
▲ Fuoco amico possibile
L'area di Nova interseca il percorso di Echo al passo 3.

i Piano modificato
Echo ha cambiato bersaglio da Guardia A a Consolle.

Il disegno sulla mappa deve usare tratti temporanei, non testo libero. Limiti proposti: massimo otto tratti attivi per giocatore, 64 punti per tratto, durata massima pari al turno, rate limit e possibilità di silenziare un singolo compagno. Questo riduce spam, abuso e costi di moderazione.

Specifica tecnica e architettura

La feature deve essere organizzata come sistema indipendente dal movimento e dalle abilità concrete. Il dominio "intent" descrive un piano; pathfinder, targeting system e conflict engine lo interpretano attraverso interfacce.



Il database non partecipa alla pianificazione live. Il server mantiene gli intenti in memoria; al termine del turno può scrivere eventi aggregati per replay, analytics e debugging. Una dipendenza sincrona da database introdurrebbe un punto di latenza e fallimento non necessario.

Modello dati

Campo	Tipo concettuale	Note
<code>IntentId</code>	<code>uint32</code> o GUID compatto	Identità stabile del piano
<code>TurnId</code>	<code>uint32</code>	Impedisce modifiche a un turno diverso
<code>Revision</code>	<code>uint32</code>	Sequenza monotona assegnata o validata dal server
<code>BaseRevision</code>	<code>uint32</code>	Base del delta; zero per snapshot completo
<code>SourcePlayerId</code>	ID server-side	Il client non può impostarlo autorevolmente
<code>TeamId</code>	Server-only	Usato per il routing, non considerato attendibile dal client
<code>IntentType</code>	<code>FGameplayTag</code>	Esempio <code>Intent.Move.Standard</code>
<code>AbilityId</code>	<code>FPrimaryAssetId</code>	Identifica definizione data-driven
<code>Path</code>	Celle o waypoint compressi	Preferire <code>CellId</code> al world-space definitivo
<code>Target</code>	Actor ID, cella o oggetto	Nessun riferimento a entità nascoste non visibili
<code>Area</code>	Shape + parametri	Cerchio, cono, linea, custom
<code>LabelTag</code>	<code>FGameplayTag</code>	Esempio <code>Intent.Label.Focus</code>
<code>PlanningState</code>	Enum	Editing, Ready, Locked
<code>FieldMask</code>	Bitmask	Campi modificati nel delta

Campo	Tipo concettuale	Note
<code>CustomPayload</code>	Struct versionata	Solo per intenti registrati
<code>ServerTimestamp</code>	Tempo match	Debug e ordinamento secondario
<code>ContentHash</code>	Hash breve	Verifica snapshot e mod manifest

I Gameplay Tag sono etichette gerarchiche definite dall'utente e adatte a classificare oggetti e concetti di gameplay. I Primary Data Asset possono essere identificati e caricati tramite Asset Manager; i Data Registry forniscono un archivio globale efficiente per dati `USTRUCT` prevalentemente read-only. Questi strumenti sono una base naturale per tipi, label e renderer moddabili. ⁷

Per il path multilivello, il payload di produzione dovrebbe contenere riferimenti alle celle:

```
struct FCellRef
{
    uint16 X;
    uint16 Y;
    uint8 Layer;
};
```

Il client può ricostruire la spline usando il centro delle celle e i collegamenti fra livelli. La conferma non deve fidarsi del path proposto: il server deve ricalcolarlo o validare ogni arco rispetto alla mappa corrente, all'unità, agli effetti ambientali e alle capacità di movimento.

Messaggi di rete

Messaggio	Direzione	Payload principale	Frequenza	Affidabilità
<code>C2S_IntentPreview</code>	Client → server	Turno, revisione, field mask, path/target/area modificati	0–12 Hz	Unreliable
<code>S2C_IntentPreview</code>	Server → alleati	Snapshot o delta canonico	0–12 Hz per autore	Unreliable
<code>C2S_IntentConfirm</code>	Client → server	Intent ID, revisione, hash	Evento	Reliable
<code>S2C_IntentConfirmed</code>	Server → alleati	Intento canonico e stato Ready	Evento	Reliable
<code>S2C_IntentRejected</code>	Server → autore	Revisione, reason code, eventuale correzione	Evento	Reliable
<code>C2S_SetPlanningState</code>	Client → server	Ready o Editing	Raro	Reliable

Messaggio	Direzione	Payload principale	Frequenza	Affidabilità
<code>S2C_PlayerPlanningState</code>	Server → squadra	Player ID e stato	Raro	Replicated/ reliable
<code>C2S_RollbackIntent</code>	Client → server	Turno e revisione sorgente	Raro	Reliable
<code>S2C_IntentSnapshot</code>	Server → client	Tutti gli intenti visibili	Join, reconnect, healing	Reliable
<code>C2S_MapStrokeChunk</code>	Client → server	Stroke ID, sequenza, punti compressi	5–10 Hz durante il tratto	Unreliable
<code>S2C_MapStrokeChunk</code>	Server → alleati	Stroke validato	5–10 Hz	Unreliable
<code>S2C_ConflictSnapshot</code>	Server → squadra	Conflitti attivi e revisioni correlate	2–10 Hz	Unreliable latest-wins
<code>S2C_TurnLocked</code>	Server → tutti	Turn ID e timestamp di risoluzione	Una volta	Reliable

Le RPC di Unreal sono chiamate unidirezionali e non restituiscono un valore, quindi conferma e rifiuto richiedono un messaggio separato. Epic indica inoltre le RPC unreliable per eventi transitori e la property replication per stato che deve convergere; una proprietà replicata arriva in modo affidabile al valore finale, ma non garantisce che il client osservi ogni valore intermedio. Questo comportamento è utile per una preview “latest wins”. ⁸

Strategia di sincronizzazione consigliata

- Il client disegna immediatamente la propria modifica.
- Ogni 80–100 ms invia l'ultimo delta disponibile.
- Il server ignora revisioni vecchie o duplicate.
- Gli alleati applicano solo revisioni superiori.
- Ogni secondo, oppure dopo inattività, il server replica lo snapshot corrente attraverso una proprietà owner-only o una Fast Array.
- Conferma e lock usano messaggi reliable.
- Dopo reconnect viene inviato uno snapshot completo.

`FFastArraySerializer` è progettato per la replica delta di array e dispone anche di integrazione con Iris; è adatto a una lista di intenti visibili sul componente owner-only del destinatario. ⁹

Autorità e visibilità selettiva

Il server deve derivare da solo `SourcePlayerId` e `TeamId`. Il client invia il contenuto della proposta, non l'identità autorevole né la lista dei destinatari.

Per quattro contro quattro, la soluzione più semplice e sicura è:

1. Ogni `PlayerController` possiede un `USharedIntentComponent`.
2. Il client invia una Server RPC sul proprio componente.
3. Il server valida e salva l'intento.
4. Il server individua i `PlayerController` della stessa squadra.
5. Chiama una Client RPC sul componente di ciascun destinatario.
6. I controller nemici non ricevono alcuna chiamata.

Un'alternativa più data-oriented è conservare, su ogni `PlayerController`, una Fast Array owner-only contenente gli intenti che quel client può vedere. I `PlayerController` sono particolarmente comodi perché ogni client possiede il proprio controller; `GameState` e `PlayerState`, invece, sono normalmente pensati per replicare informazioni condivise fra i client e non devono contenere direttamente dati tattici segreti. ¹⁰

`NetCullDistanceSquared` non deve essere trattato come controllo di privacy: è un'ottimizzazione spaziale della relevancy. Un avversario vicino potrebbe comunque diventare rilevante. Per attori team-scoped persistenti servono `bOnlyRelevantToOwner`, `IsNetRelevantFor` custom o un Replication Graph personalizzato. ¹¹

Regola fondamentale: i ghost, le spline, i decal AOE e le frecce devono essere attori o componenti **locali non replicati**. Si replica il dato compatto; ogni client genera la propria rappresentazione grafica.

Gestione dei conflitti

Il conflict engine riceve l'insieme delle revisioni canoniche e produce record come:

```
ConflictId
RuleTag
Severity
ParticipantIntentIds
TimeSlice
CellIds
MessageKey
Blocking
```

Le regole iniziali sono:

Regola	Tipo
Stessa cella finale nello stesso time slice	Warning
Percorsi che attraversano una cella esclusiva nello stesso istante	Warning
AOE alleata che interseca ghost o path alleato	Warning
Due interazioni su oggetto monouso	Warning o blocking
Target non più valido	Blocking
Path non percorribile	Blocking

Regola	Tipo
Abilità in cooldown o senza risorsa	Blocking
Cambio della mappa che invalida un arco	Blocking
Portale, ascensore o cover richiesti da più utenti	Rule-specific

Il calcolo può essere debounced di 50–100 ms. Ogni warning deve dichiarare le revisioni su cui è stato calcolato; se una di esse cambia, il warning viene eliminato o ricalcolato.

Implementazione in Unreal Engine

La divisione consigliata è netta:

Layer	Tecnologia consigliata
Struct di rete, validazione, rate limit, routing e revisioni	C++
Input di pianificazione e prototipazione	Blueprint
Ghost, spline, decal, widget e animazioni	Blueprint/UMG/Niagara
Definizioni di intenti e label	Primary Data Asset, Gameplay Tags
Conflict rules comuni	C++ con parametri data-driven
Mod data-only	Plugin di contenuto, Data Asset, JSON importato
Tool editor per creare intenti	Blueprint o C++ Editor Module

Epic consiglia Blueprints per logica ad alto livello e casi specifici, mentre il C++ offre un controllo maggiore quando servono prestazioni, matematica complessa o gestione precisa della replica. Per questo progetto il compromesso migliore è un piccolo core C++ stabile, esposto a Blueprint con `BlueprintCallable`, `BlueprintAssignable` e `BlueprintNativeEvent`. ¹²

Configurazione del sistema

Su Windows:

1. Installare Unreal Engine tramite Epic Games Launcher.
2. Installare Visual Studio compatibile con la versione di Unreal scelta e il workload **Game development with C++**. La documentazione Epic mantiene una matrice fra versione di Unreal e versione di Visual Studio; per UE 5.7 sono supportate versioni recenti di Visual Studio 2022, mentre la documentazione di UE 5.8 include anche Visual Studio 2026. ¹³
3. Creare un progetto **Games → Blank → C++**, con Starter Content opzionale.
4. Aprire `Tools → New C++ Class` almeno una volta se si parte da un progetto Blueprint; il wizard converte il progetto in un code project e genera modulo, `.h` e `.cpp`. ¹⁴
5. Aggiungere a `YourGame.Build.cs`:

```

PublicDependencyModuleNames.AddRange(
    new string[]
    {
        "Core",
        "CoreUObject",
        "Engine",
        "InputCore",
        "NetCore",
        "GameplayTags",
        "UMG"
    }
);

```

1. In **Edit → Plugins**, verificare o abilitare:
2. Gameplay Tags;
3. Data Registry, più avanti;
4. Gameplay Abilities solo se già previsto per il sistema delle abilità;
5. Functional Testing per i test.
6. Creare **BP_IntentPlayerController**, derivato dalla classe C++ del controller, e aggiungere **SharedIntentComponent**.
7. Creare **BP_IntentPlayerState**, derivato dal PlayerState C++.
8. Assegnare controller e player state nel GameMode.
9. In Play In Editor configurare quattro o più player e, quando possibile, un dedicated server. Unreal permette di testare più client, server separati, latenza e packet loss direttamente dalle opzioni PIE. 15

Per chi arriva da C#, Rider può risultare più familiare, ma resta necessario il toolchain C++ di Unreal. Epic mantiene una guida specifica per configurare Rider. 16

Codice C++ minimo

Il seguente scheletro usa RPC dirette. È intenzionalmente semplice; nella beta, lo snapshot autoritativo può essere migrato a Fast Array. Sostituire **YOURGAME_API** con il macro del progetto, per esempio **TACTICALGAME_API**.

SharedIntentTypes.h

```

#pragma once

#include "CoreMinimal.h"
#include "Engine/NetSerialization.h"
#include "SharedIntentTypes.generated.h"

UENUM(BlueprintType)
enum class EPlanningState : uint8
{
    Editing,

```

```

    Ready,
    Locked
};

USTRUCT(BlueprintType)
struct FSharedIntentData
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 TurnId = 0;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 Revision = 0;

    // Il server sovrascrive sempre questo campo.
    UPROPERTY(VisibleAnywhere, BlueprintReadOnly)
    int32 SourcePlayerId = -1;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    FName IntentType = TEXT("Move");

    // Semplice per il prototipo.
    // In produzione usare ID di celle compressi.
    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    TArray<FVector_NetQuantize10> PathPoints;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    FVector_NetQuantize10 TargetLocation = FVector::ZeroVector;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 TargetActorId = -1;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    FName Label = TEXT("Intent.Label.None");

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    EPlanningState PlanningState = EPlanningState::Editing;

    UPROPERTY(VisibleAnywhere, BlueprintReadOnly)
    bool bConfirmed = false;
};

```

IntentPlayerState.h

```

#pragma once

#include "CoreMinimal.h"
#include "GameFramework/PlayerState.h"
#include "IntentPlayerState.generated.h"

```

```

UCLASS()
class YOURGAME_API AIntentPlayerState : public APlayerState
{
    GENERATED_BODY()

public:
    UPROPERTY(Replicated, EditAnywhere, BlueprintReadOnly, Category = "Team")
    uint8 TeamId = 0;

    virtual void GetLifetimeReplicatedProps(
        TArray<FLifetimeProperty>& OutLifetimeProps
    ) const override;
};

```

IntentPlayerState.cpp

```

#include "IntentPlayerState.h"
#include "Net/UnrealNetwork.h"

void AIntentPlayerState::GetLifetimeReplicatedProps(
    TArray<FLifetimeProperty>& OutLifetimeProps
) const
{
    Super::GetLifetimeReplicatedProps(OutLifetimeProps);

    DOREPLIFETIME(AIntentPlayerState, TeamId);
}

```

SharedIntentComponent.h

```

#pragma once

#include "CoreMinimal.h"
#include "Components/ActorComponent.h"
#include "SharedIntentTypes.h"
#include "SharedIntentComponent.generated.h"

DECLARE_DYNAMIC_MULTICAST_DELEGATE_OneParam(
    FSharedIntentReceived,
    const FSharedIntentData&,
    Intent
);

UCLASS(
    ClassGroup = (Network),
    meta = (BlueprintSpawnableComponent)
)
class YOURGAME_API USharedIntentComponent : public UActorComponent

```

```

{
    GENERATED_BODY()

public:
    USharedIntentComponent();

    UPROPERTY(BlueprintAssignable, Category = "Shared Intent")
    FSharedIntentReceived OnIntentReceived;

    UFUNCTION(BlueprintCallable, Category = "Shared Intent")
    void SubmitPreview(FSharedIntentData Intent);

    UFUNCTION(BlueprintCallable, Category = "Shared Intent")
    void ConfirmIntent(FSharedIntentData Intent);

protected:
    UFUNCTION(Server, Unreliable, WithValidation)
    void ServerSubmitPreview(FSharedIntentData Intent);

    UFUNCTION(Server, Reliable, WithValidation)
    void ServerConfirmIntent(FSharedIntentData Intent);

    UFUNCTION(Client, Unreliable)
    void ClientReceivePreview(FSharedIntentData Intent);

    UFUNCTION(Client, Reliable)
    void ClientReceiveConfirmed(FSharedIntentData Intent);

private:
    bool IsStructurallyValid(const FSharedIntentData& Intent) const;

    void BroadcastToTeam(
        FSharedIntentData Intent,
        bool bReliable
    );

    int32 LastAcceptedRevision = -1;
    double LastPreviewServerTime = 0.0;
};

```

SharedIntentComponent.cpp

```

#include "SharedIntentComponent.h"

#include "IntentPlayerState.h"
#include "GameFramework/PlayerController.h"
#include "Engine/World.h"

USharedIntentComponent::USharedIntentComponent()
{

```

```

        SetIsReplicatedByDefault(true);
    }

    void USharedIntentComponent::SubmitPreview(FSharedIntentData Intent)
    {
        // La UI deve già aver disegnato la preview locale.
        ServerSubmitPreview(Intent);
    }

    void USharedIntentComponent::ConfirmIntent(FSharedIntentData Intent)
    {
        ServerConfirmIntent(Intent);
    }

    bool USharedIntentComponent::IsStructurallyValid(
        const FSharedIntentData& Intent
    ) const
    {
        if (Intent.TurnId < 0 || Intent.Revision < 0)
        {
            return false;
        }

        if (Intent.PathPoints.Num() > 32)
        {
            return false;
        }

        for (const FVector_NetQuantize10& Point : Intent.PathPoints)
        {
            if (FVector(Point).ContainsNaN())
            {
                return false;
            }
        }

        return !FVector(Intent.TargetLocation).ContainsNaN();
    }

    bool USharedIntentComponent::ServerSubmitPreview_Validate(
        FSharedIntentData Intent
    )
    {
        // Solo controlli strutturali.
        // Non disconnettere il client per una normale revisione vecchia.
        return IsStructurallyValid(Intent);
    }

    void USharedIntentComponent::ServerSubmitPreview_Implementation(
        FSharedIntentData Intent
    )

```



```

{
    const double Now = FPlatformTime::Seconds();

    // Massimo circa 12 update al secondo.
    if (Now - LastPreviewServerTime < 0.08)
    {
        return;
    }

    LastPreviewServerTime = Now;

    // Ignora pacchetti vecchi o duplicati.
    if (Intent.Revision <= LastAcceptedRevision)
    {
        return;
    }

    // TODO:
    // - verificare TurnId contro il GameState;
    // - validare path e target;
    // - ricalcolare il path canonico.

    Intent.bConfirmed = false;
    Intent.PlanningState = EPlanningState::Editing;

    LastAcceptedRevision = Intent.Revision;
    BroadcastToTeam(Intent, false);
}

bool USharedIntentComponent::ServerConfirmIntent_Validate(
    FSharedIntentData Intent
)
{
    return IsStructurallyValid(Intent);
}

void USharedIntentComponent::ServerConfirmIntent_Implementation(
    FSharedIntentData Intent
)
{
    // È permesso confermare la stessa revisione già inviata come preview.
    if (Intent.Revision < LastAcceptedRevision)
    {
        return;
    }

    // TODO:
    // - verificare il turno;
    // - ricalcolare il path;
    // - validare abilità, risorse, target e mappa;
    // - salvare lo snapshot canonico.
}

```

```

Intent.bConfirmed = true;
Intent.PlanningState = EPlanningState::Ready;

LastAcceptedRevision = Intent.Revision;
BroadcastToTeam(Intent, true);
}

void USharedIntentComponent::BroadcastToTeam(
    FSharedIntentData Intent,
    bool bReliable
)
{
    APlayerController* SenderController =
        Cast<APlayerController>(GetOwner());

    if (!SenderController || !GetWorld())
    {
        return;
    }

    AIntentPlayerState* SenderState =
        SenderController->GetPlayerState<AIntentPlayerState>();

    if (!SenderState)
    {
        return;
    }

    // Non fidarsi mai del SourcePlayerId mandato dal client.
    Intent.SourcePlayerId = SenderState->GetPlayerId();

    for (
        FConstPlayerControllerIterator It =
            GetWorld()->GetPlayerControllerIterator();
        It;
        ++It
    )
    {
        APlayerController* RecipientController = It->Get();

        if (!RecipientController)
        {
            continue;
        }

        AIntentPlayerState* RecipientState =
            RecipientController->GetPlayerState<AIntentPlayerState>();

        if (!RecipientState ||
            RecipientState->TeamId != SenderState->TeamId)

```

```

    {
        continue;
    }

    USharedIntentComponent* RecipientComponent =
        RecipientController
            ->FindComponentByClass<USharedIntentComponent>();

    if (!RecipientComponent)
    {
        continue;
    }

    if (bReliable)
    {
        RecipientComponent->ClientReceiveConfirmed(Intent);
    }
    else
    {
        RecipientComponent->ClientReceivePreview(Intent);
    }
}
}

void USharedIntentComponent::ClientReceivePreview_Implementation(
    FSharedIntentData Intent
)
{
    OnIntentReceived.Broadcast(Intent);
}

void USharedIntentComponent::ClientReceiveConfirmed_Implementation(
    FSharedIntentData Intent
)
{
    OnIntentReceived.Broadcast(Intent);
}
}

```

Il codice applica la proprietà più importante: **il filtro di squadra avviene sul server**. Il client nemico non riceve l'RPC. La validazione delle Server RPC è supportata direttamente dagli specifier `Server`, `Reliable` / `Unreliable` e `WithValidation` di Unreal. ¹⁷

Per una build reale, aggiungere:

- controllo del `TurnId` contro un GameState autoritativo;
- rate limiter a token bucket;
- limite in byte del payload;
- path canonico server-side;
- reason code di rifiuto;
- acknowledgement all'autore;

- snapshot di healing;
- confronto fra hash del contenuto e revisione;
- log di sicurezza;
- filtri sulle entità visibili;
- test automatici di assenza di leakage.

Blueprint pseudo-code

Aggiornamento della traiettoria

```
Event IA_PlanMove Triggered
  → Get Mouse World Position
  → Find Path For Selected Character
  → Build SharedIntentData
    TurnId = CurrentTurn
    Revision = Revision + 1
    IntentType = "Move"
    PathPoints = Path
    TargetLocation = LastPathPoint
  → Render Local Ghost Immediately
  → Store as PendingIntent
```

Invio con throttle

```
Timer ogni 0.10 secondi
  → Branch: PendingIntent è cambiato?
    True
      → SharedIntentComponent.SubmitPreview(PendingIntent)
      → Set PendingIntentChanged = False
```

Ricezione di un intento alleato

```
Event OnIntentReceived(Intent)
  → Find or Spawn Local IntentGhost by SourcePlayerId
  → Set Ghost Path
  → Set Target Marker
  → Set AOE Shape
  → Set Label Icon
  → Set Ready Visual from Intent.PlanningState
```

Conferma

```
On Confirm Button
  → PendingIntent.PlanningState = Ready
```

```
→ SharedIntentComponent.ConfirmIntent(PendingIntent)
→ Show "Conferma in corso"
```

La UI non deve applicare un ulteriore filtro di sicurezza per squadra: può farlo come controllo difensivo, ma la garanzia reale resta il routing server-side.

NetMulticast, NetCull e Replication Graph

`NetMulticast` può essere usato per eventi pubblici, come l'animazione di inizio risoluzione, ma non per inviare dati tattici privati. Le RPC multicast vengono eseguite dal server e dai client ai quali l'attore viene replicato; trasformare la relevancy in un sistema di autorizzazione tattica aumenta il rischio di errori. ¹⁸

`NetCullDistanceSquared` è utile per elementi pubblici vicini, non per gli intenti. Le intenzioni di un alleato devono essere visibili anche dall'altro lato della mappa e invisibili a un nemico adiacente.

Un Replication Graph custom è giustificato soltanto se il progetto cresce verso molte squadre, spettatori con permessi diversi o decine di giocatori. Per un quattro contro quattro, Client RPC mirate e componenti owner-only sono molto più semplici da verificare.

C# e plugin bridge

UnrealCLR integra un runtime .NET e dichiara supporto a C# e .NET; UnrealSharp è un altro progetto open source attivo che permette di creare classi Unreal in C# e supporta hot reload. Entrambi sono strumenti di terze parti, non parte del toolchain ufficiale di Epic. ¹⁹

La raccomandazione è:

Uso	C++ minimo	C# bridge
RPC e autorità server	Raccomandato	Evitare nel core iniziale
Struct replicate	Raccomandato	Solo dopo spike tecnico
Renderer di ghost	Blueprint	Possibile
Tool editor	Blueprint/C++	Buon candidato
Utility di import JSON	C++ o Blueprint	Buon candidato
Dedicated server	Baseline ufficiale	Verificare packaging e piattaforme
ABI pubblica per mod	Interfacce Unreal	Non dipendere dal bridge

La scelta safe è imparare il sottoinsieme di C++ necessario a Unreal: `UCLASS`, `USTRUCT`, `UPROPERTY`, `UFUNCTION`, puntatori gestiti da `UPROPERTY`, componenti e RPC. La logica visiva può restare in Blueprint. Un bridge C# può essere sperimentato in un branch separato, senza renderlo una dipendenza del protocollo multiplayer.

Test, metriche e criteri di accettazione

Unreal fornisce Network Emulation per simulare latenza e perdita di pacchetti, Networking Insights per analizzare il traffico e Automation/Functional Testing per test di basso e alto livello. Gauntlet può avviare sessioni che includono server e più client ed è adatto a test multiplayer automatizzati. ²⁰

Piano di test

Livello	Test principali
Unit test	Serializzazione, confronto revisioni, validazione payload, compressione path, regole di conflitto
Component test	Throttle, rate limiter, creazione snapshot, rollback, eliminazione cronologia
Integration test	Client → server → soli alleati, conferma, reject, reconnect, timeout
Security test	Client modifica SourcePlayerId, TeamId, TurnId, target, dimensione array o frequenza
Visibility test	Packet capture su client nemico; nessun payload o actor reference degli intenti avversari
Network test	50–250 ms RTT, jitter, 1–10% packet loss, riordinamento e disconnect
Load test	Otto e sedici client che aggiornano simultaneamente path e disegni
Mod test	Manifest compatibile, incompatibile, hash errato, intento sconosciuto, renderer mancante
UX test	Comprensione piano, tempo per risolvere conflitti, leggibilità con overlay sovrapposti
Accessibility test	Deuteranopia/protanopia, scala UI, overlay senza colori, controller e mouse
Soak test	Match ripetuti per almeno 30–60 minuti senza crescita non controllata di cache o ghost

La Network Emulation di Unreal consente di configurare separatamente lag e packet loss in entrata e uscita, anche nelle opzioni multiplayer di Play In Editor. Networking Insights permette di ispezionare pacchetti, RPC, proprietà e traffico nel tempo. ²¹

Target tecnici

Metrica	Target alpha	Target release
Risposta della preview locale	< 33 ms	< 16,7 ms a 60 FPS
Autore → alleato, mediana	< 150 ms	< 100 ms
Autore → alleato, p95 con RTT ≤ 120 ms	< 250 ms	< 200 ms
Frequenza preview client	≤ 10 Hz	≤ 12 Hz

Metrica	Target alpha	Target release
Banda tipica per client	< 8 KB/s	< 5 KB/s
Banda di picco per client	< 15 KB/s	< 10 KB/s
Elaborazione server per update, otto player	< 2 ms	< 1 ms
Convergenza dopo perdita pacchetto	< 1,5 s	< 500 ms
Payload massimo preview	2 KB	1 KB
Path massimo	48 punti	32 punti o celle compresse
Leakage verso nemici	0	0
Crash/desync durante soak	0	0
Conflitti hard non rilevati	< 5%	< 1%

I target di latenza non derivano da un limite imposto da Unreal: sono obiettivi di prodotto. La ricerca sul contesto visivo condiviso indica però che aggiornamenti ritardati riducono i benefici collaborativi, quindi la latenza dell'overlay deve essere trattata come metrica UX, non solo di rete. ²²

Metriche di gameplay e UX

Durante playtest controllati, confrontare una build con intenti condivisi e una build con soli ping:

Metrica	Segnale desiderato
Conflitti fra alleati per turno	Riduzione
Fuoco amico non intenzionale	Riduzione
Doppie interazioni sulla stessa risorsa	Riduzione
Tempo medio per raggiungere Ready	Stabile o inferiore
Modifiche negli ultimi tre secondi	Riduzione
Ping generici per turno	Riduzione
Messaggi chat necessari per coordinare un focus	Riduzione
Accuratezza nel descrivere il piano alleato	Aumento
Percezione di clutter	Non oltre soglia definita
Utilizzo dei filtri overlay	Individuazione delle fonti di clutter
Rollback per turno	Misurare, senza assumere che meno sia sempre meglio
Win-rate gruppi con voice vs senza voice	Riduzione del divario, senza annullarlo

Criteri di accettazione

La feature è accettata per la beta quando:

- tutti gli alleati vedono traiettoria, target, AOE, label e stato Ready;
- un nemico non riceve alcuna intenzione avversaria, verificato tramite log e packet capture;
- pacchetti preview fuori ordine non riportano l'overlay a una revisione precedente;
- la perdita di un update non impedisce la convergenza allo stato più recente;
- conferma e lock arrivano in modo affidabile;
- un client non può cambiare `SourcePlayerId`, `TeamId` o destinatari;
- un client non può confermare un turno già chiuso;
- path e target vengono validati dal server;
- modificare una mossa dopo Ready riporta correttamente il giocatore in Editing;
- il rollback crea una nuova revisione monotona;
- al reconnect il client ricostruisce tutti gli intenti alleati correnti;
- i warning di conflitto indicano partecipanti, area e revisione;
- l'utente può nascondere disegni, AOE o singoli compagni senza influenzare il gameplay;
- lo stato bloccato usato dal resolver è immutabile;
- il sistema regge otto client con 5% packet loss e 150 ms RTT senza desync permanente;
- le mod non approvate dal server non possono registrare payload o regole di rete.

Roadmap, modding, rischi e risorse

Le stime seguenti assumono un programmatore full-time, UI provvisoria e disponibilità dei sistemi dipendenti. Con sviluppo part-time, il calendario si estende in proporzione.

Roadmap

Milestone	Deliverable	Dipendenze	Stima
Fondazioni	Turn state machine, team ID, PlayerController/ PlayerState custom, struct intent, test PIE a quattro client	Sessione multiplayer e GameMode	7-10 giorni
Alpha tecnica	Traiettoria, target, AOE semplice, RPC team-only, preview locale, conferma e Ready	Pathfinder e ability targeting minimi	18-24 giorni
Alpha giocabile	Ghost, overlay squadra, lock del turno, snapshot autoritativo, reject e timeout	Resolver del turno	10-15 giorni
Beta rete	Delta/revisioni, snapshot healing, reconnect, rate limiting, Networking Insights	Dedicated server stabile	10-15 giorni
Beta gameplay	Conflict engine, notifiche, cronologia, rollback, disegno mappa	Timeline del movimento e shape AOE	15-22 giorni
Feature mod finale	Registry pubblico, Gameplay Tags, Data Asset, import JSON, manifest e whitelist	Schema protocollo congelato	10-15 giorni
Polish	Accessibilità, performance, anti-spam, telemetria, test Gauntlet, UX pass	Tutte le milestone precedenti	10-15 giorni

Totale realistico: circa **80–116 giorni-persona**, equivalenti a **16–23 settimane full-time** per una singola persona. Un MVP ridotto senza disegno, cronologia avanzata, rollback e modding può essere completato in circa **7–10 settimane**.

Le dipendenze critiche sono:

- identificazione affidabile delle squadre;
- state machine Planning → Locked → Resolution;
- pathfinder multilivello;
- sistema di targeting e shape AOE;
- ID stabili delle celle e degli attori;
- regole di visibilità del fog of war;
- resolver temporale per valutare collisioni e fuoco amico;
- dedicated server e reconnect.

API di modding

Unreal supporta plugin contenenti codice e dati; i Primary Data Asset possono essere caricati tramite Asset Manager e i Gameplay Tag possono essere definiti anche da file di configurazione dei plugin. Game Features e Modular Gameplay permettono di attivare funzionalità modulari, ma la documentazione corrente li segnala ancora come sistemi da utilizzare con cautela nelle build distribuite.

23

L'API pubblica dovrebbe esporre:

```
class IIntentTypeProvider;  
class IIntentRenderer;  
class IIntentConflictRule;  
class IIntentPayloadValidator;  
class IIntentDescriptionProvider;
```

Ogni intento registrato deve dichiarare:

- ID e versione;
- tag gerarchico;
- payload ammesso;
- dimensione massima;
- renderer;
- validatore server;
- regole di conflitto;
- frequenza massima;
- policy di visibilità;
- icona e chiavi di localizzazione;
- compatibilità con replay e spectator.

Esempio JSON: nuova etichetta

```
{  
  "schemaVersion": 1,
```

```

"contentType": "IntentLabel",
"id": "mod.rescue.label.cover_me",
>tag": "Intent.Label.CoverMe",
"displayNameKey": "label.cover_me.name",
"descriptionKey": "label.cover_me.description",
"icon": "/RescueMod/UI/T_Intent_CoverMe",
"sortOrder": 40,
"allowedModes": [
  "Casual",
  "Custom"
]
}

```

Esempio JSON: nuovo tipo di intento

```

{
  "schemaVersion": 1,
  "contentType": "IntentType",
  "id": "mod.breach.intent.remote_detonation",
  "version": "1.0.0",
  "tag": "Intent.Interact.RemoteDetonation",
  "renderer": "IntentRenderer.TargetAndRadius",
  "validator": "IntentValidator.RegisteredInteractable",
  "visibility": "TeamOnly",
  "maxUpdateHz": 8,
  "maxPayloadBytes": 256,
  "payload": {
    "fields": [
      {
        "name": "targetObjectId",
        "type": "NetObjectId",
        "required": true
      },
      {
        "name": "radiusCells",
        "type": "UInt8",
        "min": 1,
        "max": 12
      }
    ]
  },
  "conflictRules": [
    "Conflict.ExclusiveInteractable",
    "Conflict.FriendlyFireArea"
  ]
}

```

In ranked multiplayer, il server deve accettare soltanto manifest firmati o inclusi nella propria whitelist. Tutti i client devono possedere la stessa versione delle mod che influenzano simulazione, intenti o

conflitti. Una mod puramente cosmetica può essere client-side solo se non modifica dimensione, posizione o semantica dell'overlay in modo competitivo.

I plugin di codice nativo hanno accesso ampio al processo e non devono essere scaricati ed eseguiti automaticamente da server sconosciuti. La prima versione del modding dovrebbe quindi consentire:

- nuove label;
- icone;
- colori e pattern;
- shape basate su renderer già approvati;
- nuovi intenti composti da payload dichiarativi;
- conflict rule parametrizzate già presenti nel gioco.

Il codice C++ arbitrario rappresenta una fase successiva e separata.

Rischi e mitigazioni

Rischio	Impatto	Mitigazione
Leakage degli intenti ai nemici	Critico	Routing server-side per destinatario, packet capture automatica, nessun multicast privato
Client che falsifica identità o team	Critico	Identità derivata dal PlayerController/PlayerState server-side
Spam di RPC	Alto	Token bucket, frequenza massima, payload cap, mute temporaneo
Coda reliable congestionata	Alto	Preview unreliable; reliable solo per eventi finali
Overlay illeggibile	Alto	Filtri, opacità, focus mode, offset delle spline, pattern
Preview interpretata come risultato certo	Alto	Linguaggio visivo "ghost", warning e animazioni non definitive
Conflitti con falsi positivi	Medio	Warning non bloccanti e rule ID spiegabile
Path server diverso dal client	Alto	Canonical path nell'ack; correzione visuale breve
Pacchetto finale preview perso	Medio	Snapshot healing e conferma reliable
Rollback troppo complesso	Medio	Revision rollback, non rollback completo della simulazione
Disegni tossici o spam	Medio	Nessun testo libero, limiti, TTL, mute e report
Mod che altera il vantaggio competitivo	Critico	Manifest hash, whitelist, server authority, mode separation
Dipendenza da bridge C#	Alto	Core ufficiale C++/Blueprint; bridge isolato e opzionale
Schema mod instabile	Alto	Modding sviluppato per ultimo, dopo protocol freeze
Database indisponibile	Basso	DB fuori dal percorso critico

Rischio	Impatto	Mitigazione
Reconnect durante il lock	Medio	Snapshot immutabile server-side e modalità spettatore temporanea

Risorse prioritarie

Priorità	Risorsa	Utilità
Essenziale	Networking Overview di Unreal ²⁴	Modello autoritativo client-server
Essenziale	Remote Procedure Calls ²⁵	Server, Client, NetMulticast, reliability e validazione
Essenziale	Replicate Actor Properties ²⁶	<code>Replicated</code> , <code>ReplicatedUsing</code> e condizioni
Essenziale	Actor Relevancy ²⁷	Owner relevancy, distanza e personalizzazione
Essenziale	Multiplayer Programming Quick Start ²⁸	Primo progetto multiplayer C++
Essenziale	Testing Multiplayer e Network Emulation ²⁹	Test multi-client, lag e packet loss
Essenziale	Networking Insights ³⁰	Profilazione di RPC, proprietà e banda
Alta	Automation e Gauntlet ³¹	Test automatici con server e client
Alta	Gameplay Tags ³²	Classificazione data-driven di intenti e label
Alta	Data Assets e Asset Manager ³³	Definizioni caricabili e moddabili
Alta	Data Registries ³⁴	Registro globale dei tipi di intento
Alta	Plugins e Game Features ³⁵	Packaging modulare delle mod
Utile	Lyra Sample Game ³⁶	Esempio Epic di architettura modulare multiplayer
UX	Frozen Synapse ³⁷	Path, ordini e preview nella pianificazione
UX	Phantom Brigade ³⁸	Timeline, ghost e leggibilità delle azioni
UX	Workspace awareness research ³⁹	Principi per presenza e coordinamento visuale
Futuro	Network Prediction e Mover ⁴⁰	Rollback e resimulazione della simulazione
Futuro	GGPO SDK ⁴¹	Pattern di save, load e rollback deterministico
Opzionale	UnrealCLR ⁴²	Esperimento di integrazione .NET
Opzionale	UnrealSharp ⁴³	Alternativa C# attiva, non ufficiale

La decisione architetturale più importante è mantenere separati **dato autoritativo**, **trasporto di rete** e **rappresentazione grafica**. Il server conserva e filtra gli intenti; la rete trasporta revisioni compatte;

ogni client costruisce localmente spline, AOE e ghost. Questa separazione protegge la privacy tattica, limita la banda, semplifica il modding e permette di migliorare la UI senza cambiare il protocollo del match.

1 Awareness and coordination in shared workspaces

https://dl.acm.org/doi/pdf/10.1145/143457.143468?utm_source=chatgpt.com

2 5 37 Frozen Synapse - Game

https://www.matrixgames.com/product/frozen-synapse?utm_source=chatgpt.com

3 6 8 25 Remote Procedure Calls in Unreal Engine

https://dev.epicgames.com/documentation/unreal-engine/remote-procedure-calls-in-unreal-engine?utm_source=chatgpt.com

4 GGPO | Rollback Networking SDK for Peer-to-Peer Games

https://www.ggpo.net/?utm_source=chatgpt.com

7 32 Using Gameplay Tags in Unreal Engine

https://dev.epicgames.com/documentation/unreal-engine/using-gameplay-tags-in-unreal-engine?utm_source=chatgpt.com

9 Unreal Python 5.6 (Experimental) documentation

https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/class/FastArraySerializer?application_version=5.6&utm_source=chatgpt.com

10 Gameplay Framework in Unreal engine

https://dev.epicgames.com/documentation/unreal-engine/gameplay-framework-in-unreal-engine?utm_source=chatgpt.com

11 27 Actor Relevancy in Unreal Engine

https://dev.epicgames.com/documentation/unreal-engine/actor-relevancy-in-unreal-engine?utm_source=chatgpt.com

12 Balancing Blueprint and C++ | Unreal Engine 4.27 ...

https://dev.epicgames.com/documentation/en-us/unreal-engine/balancing-blueprint-and-cplusplus?application_version=4.27&utm_source=chatgpt.com

13 Setting Up Visual Studio

https://dev.epicgames.com/documentation/unreal-engine/setting-up-visual-studio-development-environment-for-cplusplus-projects-in-unreal-engine?utm_source=chatgpt.com

14 CPP Only Example | Unreal Engine 5.8 Documentation

https://dev.epicgames.com/documentation/unreal-engine/cpp-only-example?utm_source=chatgpt.com

15 29 Testing Multiplayer in Unreal Engine

https://dev.epicgames.com/documentation/unreal-engine/testing-multiplayer-in-unreal-engine?utm_source=chatgpt.com

16 Rider Setup Guide | Unreal Engine 5.8 Documentation

https://dev.epicgames.com/documentation/unreal-engine/rider-setup-guide?utm_source=chatgpt.com

17 UFunctions in Unreal Engine

https://dev.epicgames.com/documentation/unreal-engine/ufunctions-in-unreal-engine?utm_source=chatgpt.com

18 Networking Overview | Unreal Engine 4.27 Documentation

https://dev.epicgames.com/documentation/unreal-engine/networking-overview?application_version=4.27&utm_source=chatgpt.com

19 42 nxrighthere/UnrealCLR: Unreal Engine .NET 6 integration

https://github.com/nxrighthere/UnrealCLR?utm_source=chatgpt.com

20 **Using Network Emulation in Unreal Engine**

[https://dev.epicgames.com/documentation/unreal-engine/using-network-emulation-in-unreal-engine?](https://dev.epicgames.com/documentation/unreal-engine/using-network-emulation-in-unreal-engine?utm_source=chatgpt.com)
[utm_source=chatgpt.com](https://dev.epicgames.com/documentation/unreal-engine/using-network-emulation-in-unreal-engine?utm_source=chatgpt.com)

21 **Play In Editor Multiplayer Options in Unreal Engine**

[https://dev.epicgames.com/documentation/unreal-engine/play-in-editor-multiplayer-options-in-unreal-engine?](https://dev.epicgames.com/documentation/unreal-engine/play-in-editor-multiplayer-options-in-unreal-engine?utm_source=chatgpt.com)
[utm_source=chatgpt.com](https://dev.epicgames.com/documentation/unreal-engine/play-in-editor-multiplayer-options-in-unreal-engine?utm_source=chatgpt.com)

22 **The Use of Visual Information in Shared Visual Spaces**

https://sfussell.hci.cornell.edu/pubs/Manuscripts/p318-kraut.pdf?utm_source=chatgpt.com

23 35 **Plugins in Unreal Engine**

https://dev.epicgames.com/documentation/unreal-engine/plugins-in-unreal-engine?utm_source=chatgpt.com

24 **Networking Overview for Unreal Engine**

https://dev.epicgames.com/documentation/unreal-engine/networking-overview-for-unreal-engine?utm_source=chatgpt.com

26 **Replicate Actor Properties in Unreal Engine**

[https://dev.epicgames.com/documentation/unreal-engine/replicate-actor-properties-in-unreal-engine?](https://dev.epicgames.com/documentation/unreal-engine/replicate-actor-properties-in-unreal-engine?utm_source=chatgpt.com)
[utm_source=chatgpt.com](https://dev.epicgames.com/documentation/unreal-engine/replicate-actor-properties-in-unreal-engine?utm_source=chatgpt.com)

28 **Multiplayer Programming Quick Start for Unreal Engine**

[https://dev.epicgames.com/documentation/unreal-engine/multiplayer-programming-quick-start-for-unreal-engine?](https://dev.epicgames.com/documentation/unreal-engine/multiplayer-programming-quick-start-for-unreal-engine?utm_source=chatgpt.com)
[utm_source=chatgpt.com](https://dev.epicgames.com/documentation/unreal-engine/multiplayer-programming-quick-start-for-unreal-engine?utm_source=chatgpt.com)

30 **Networking Insights in Unreal Engine**

https://dev.epicgames.com/documentation/unreal-engine/networking-insights-in-unreal-engine?utm_source=chatgpt.com

31 **Automation Test Framework in Unreal Engine**

[https://dev.epicgames.com/documentation/unreal-engine/automation-test-framework-in-unreal-engine?](https://dev.epicgames.com/documentation/unreal-engine/automation-test-framework-in-unreal-engine?utm_source=chatgpt.com)
[utm_source=chatgpt.com](https://dev.epicgames.com/documentation/unreal-engine/automation-test-framework-in-unreal-engine?utm_source=chatgpt.com)

33 **Data Assets in Unreal Engine**

https://dev.epicgames.com/documentation/unreal-engine/data-assets-in-unreal-engine?utm_source=chatgpt.com

34 **Data Registries in Unreal Engine**

https://dev.epicgames.com/documentation/unreal-engine/data-registries-in-unreal-engine?utm_source=chatgpt.com

36 **Lyra Sample Game in Unreal Engine**

https://dev.epicgames.com/documentation/unreal-engine/lyra-sample-game-in-unreal-engine?utm_source=chatgpt.com

38 **Phantom Brigade**

https://braceyourselfgames.com/phantom-brigade/?utm_source=chatgpt.com

39 **A Descriptive Framework of Workspace Awareness for ...**

[https://collablab.northwestern.edu/CollablabDistro/nucmc/GutwinGreenberg_FrameworkWorkspaceAwareness.pdf?](https://collablab.northwestern.edu/CollablabDistro/nucmc/GutwinGreenberg_FrameworkWorkspaceAwareness.pdf?utm_source=chatgpt.com)
[utm_source=chatgpt.com](https://collablab.northwestern.edu/CollablabDistro/nucmc/GutwinGreenberg_FrameworkWorkspaceAwareness.pdf?utm_source=chatgpt.com)

40 **NetworkPrediction | Unreal Engine 5.8 Documentation**

https://dev.epicgames.com/documentation/unreal-engine/API/Plugins/NetworkPrediction?utm_source=chatgpt.com

41 **pond3r/ggpo: Good Game, Peace Out Rollback Network SDK**

https://github.com/pond3r/ggpo?utm_source=chatgpt.com

43 **UnrealSharp is a plugin to Unreal Engine 5, which enables ...**

https://github.com/UnrealSharp/UnrealSharp?utm_source=chatgpt.com